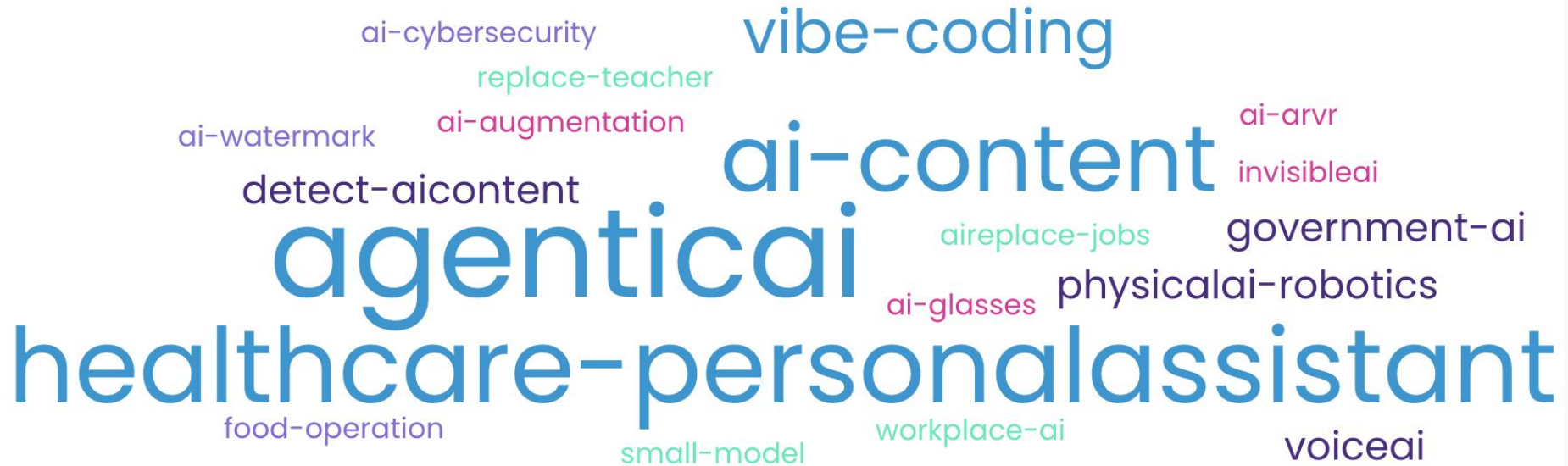


A1: your prediction for 2026



A1: not happening in 2026

invisibleai

nuclear-powered

agenticalai

energy-efficient

disrupt-jobs

sustainable-ai

dominant-aicontent

ai-robots

Quiz#3 Convolution & MaxPooling

Given an input image of width x height = 256x256, apply convolution with a 7x7 kernel, using padding=2 and stride=1. What will be the resolution of the output?

- ☐ 257x257
- ☐ 253x253
- ☐ 255x255
- ☐ 256x256
- ☐ 254x254

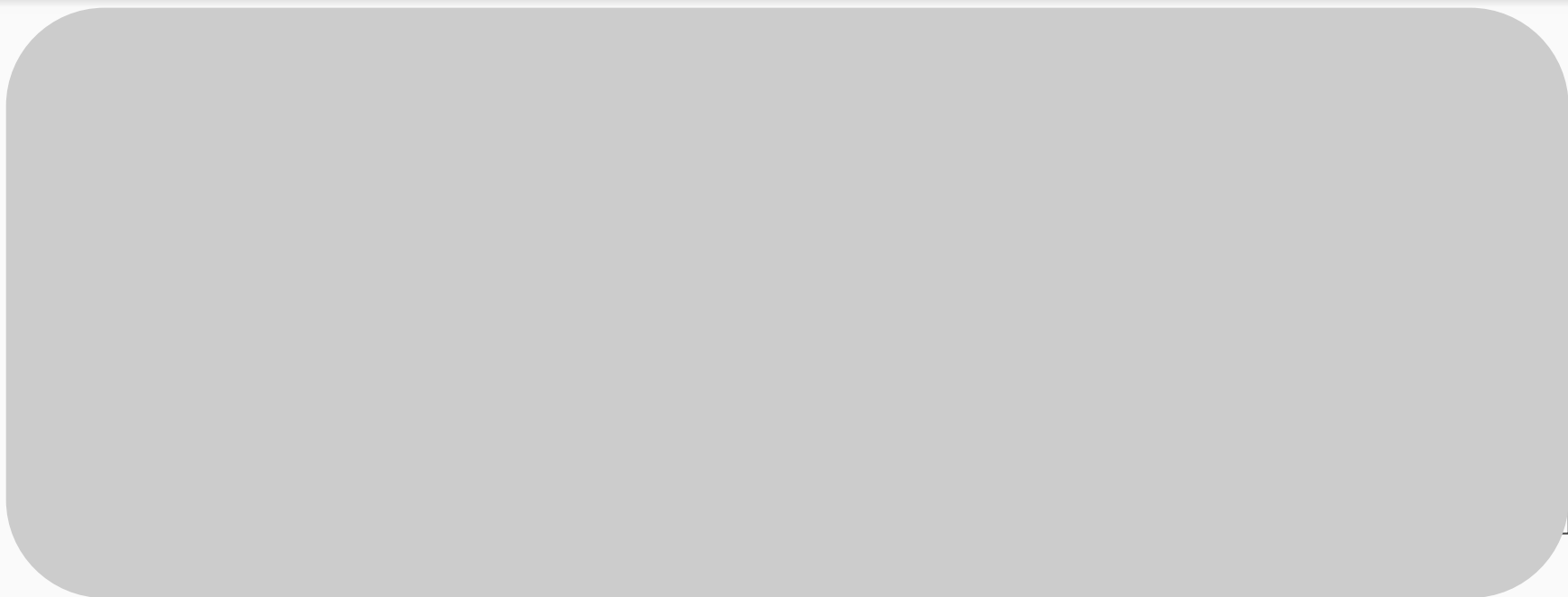
Canvas → Modules → Quizzes

Given an input image of width x height = 128x128, apply max pooling with a 4x4 kernel, no padding and stride=2. What will be the resolution of the output?

- ☐ 32x32
- ☐ 64x64
- ☐ 63x63
- ☐ 31x31
- ☐ 47x47
- ☐ 48x48

Reference Answer

– N: input size, K: kernel size, P: padding, S: stride



CNN Advanced Concept

CSCI 611 Sp25 Week 4 Session 1

Bo Shen

Convolution & Max Pooling - Space Invariant

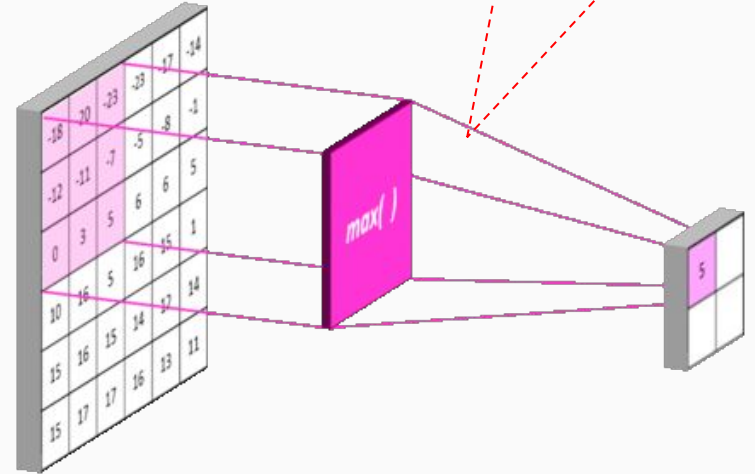
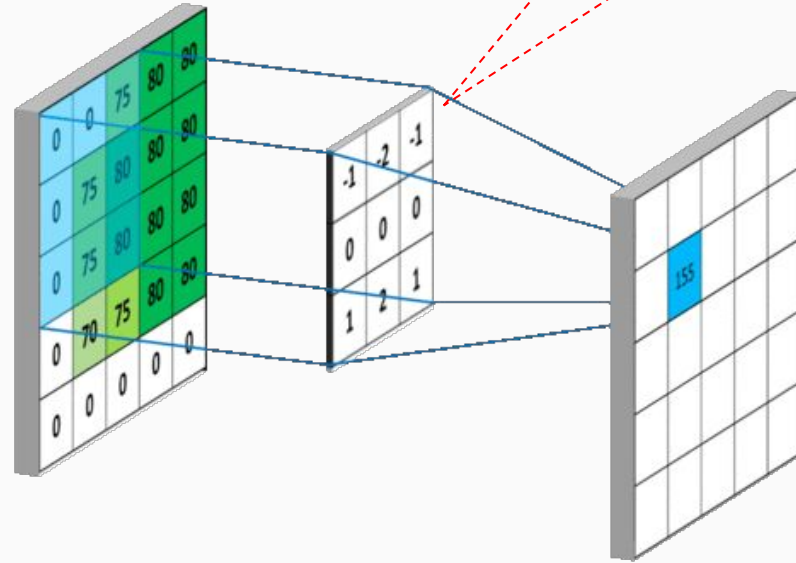
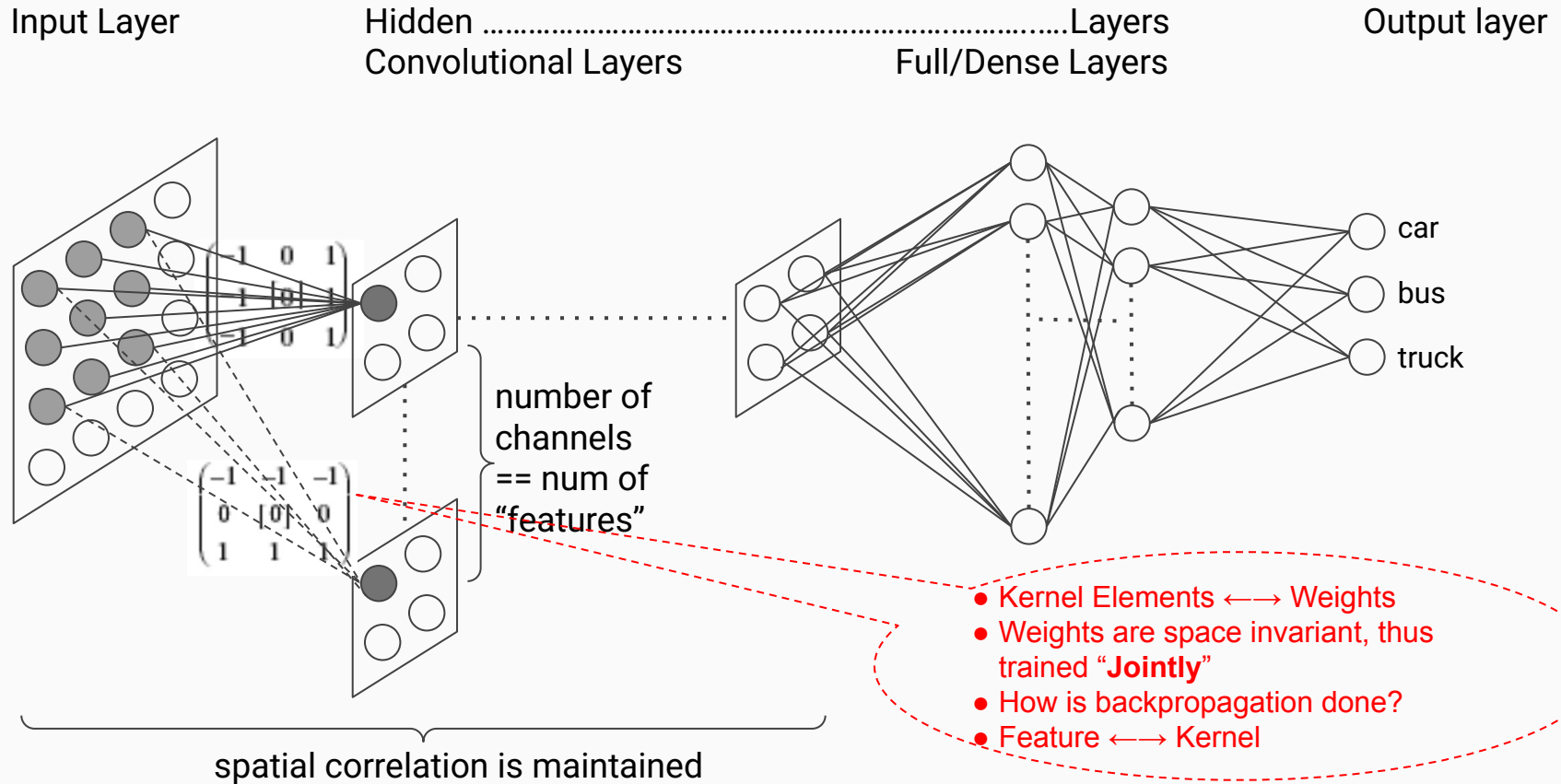
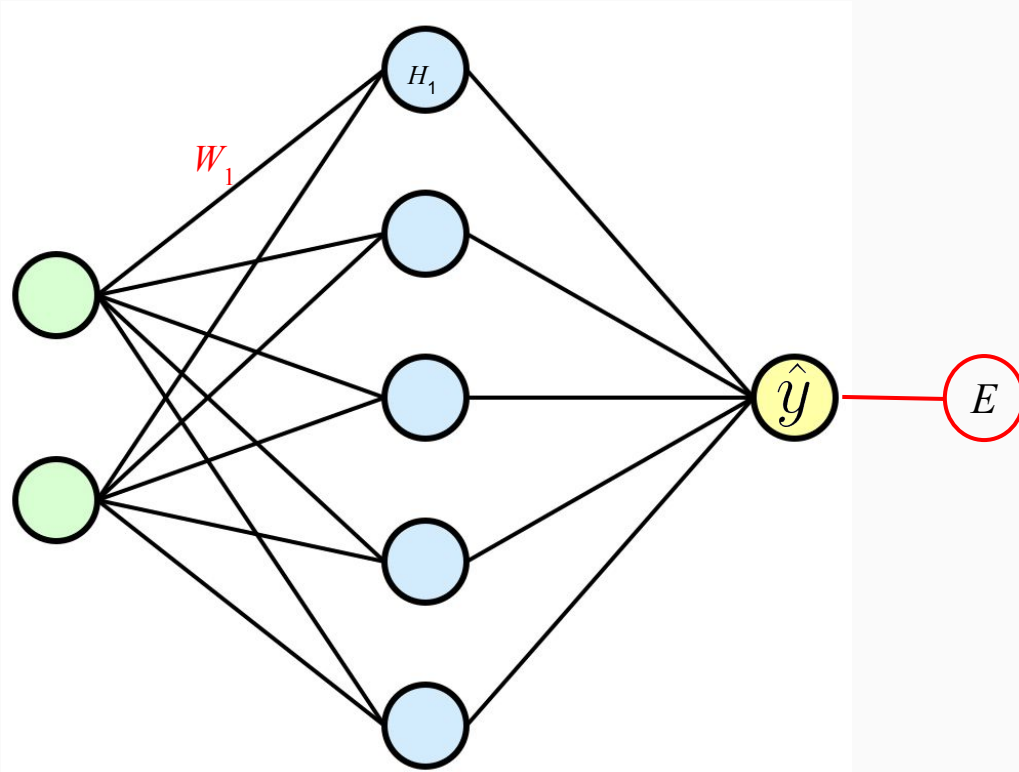


Image Filtering Kernel ~ Weights in CNN



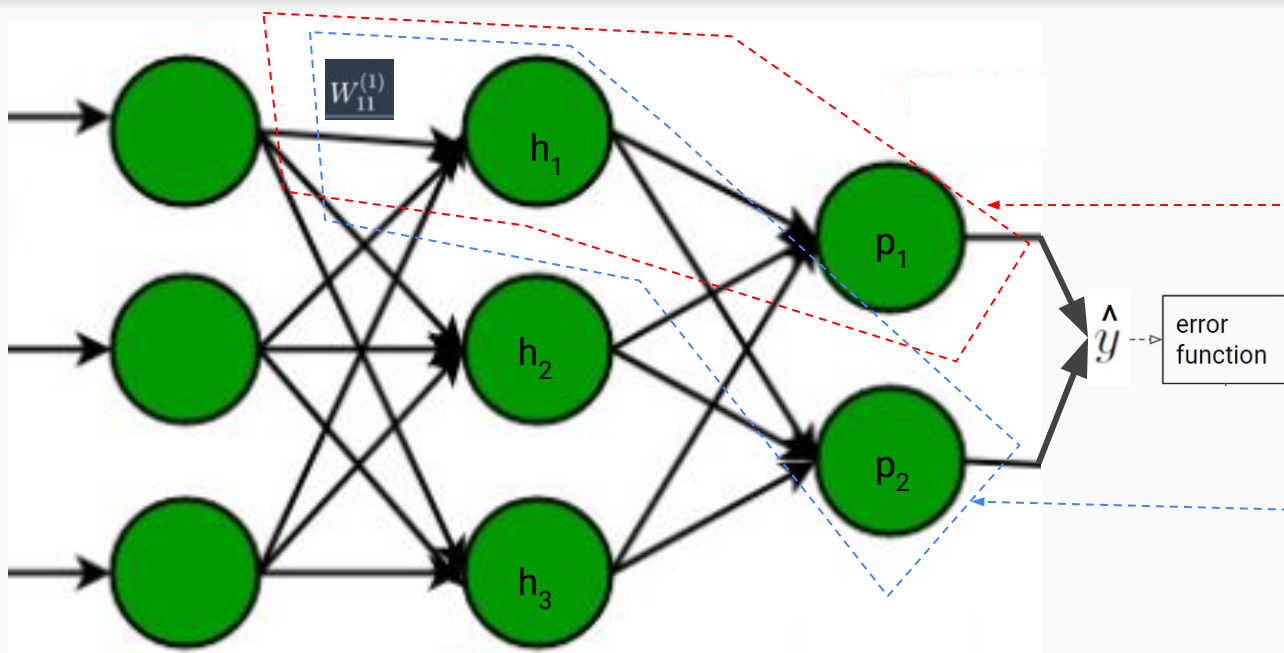
Backpropagation **One to One** - Weight Connects One Node to One Node



$$\frac{\delta E}{\delta W_1} = \frac{\delta E}{\delta \hat{y}} \frac{\delta \hat{y}}{\delta H_1} \frac{\delta H_1}{\delta W_1}$$

Multi-Layer Backpropagation

- **One to Many** (A weight connects one node to multiple nodes)
- **Many to One** (Multiple nodes can backpropagate to one weight)



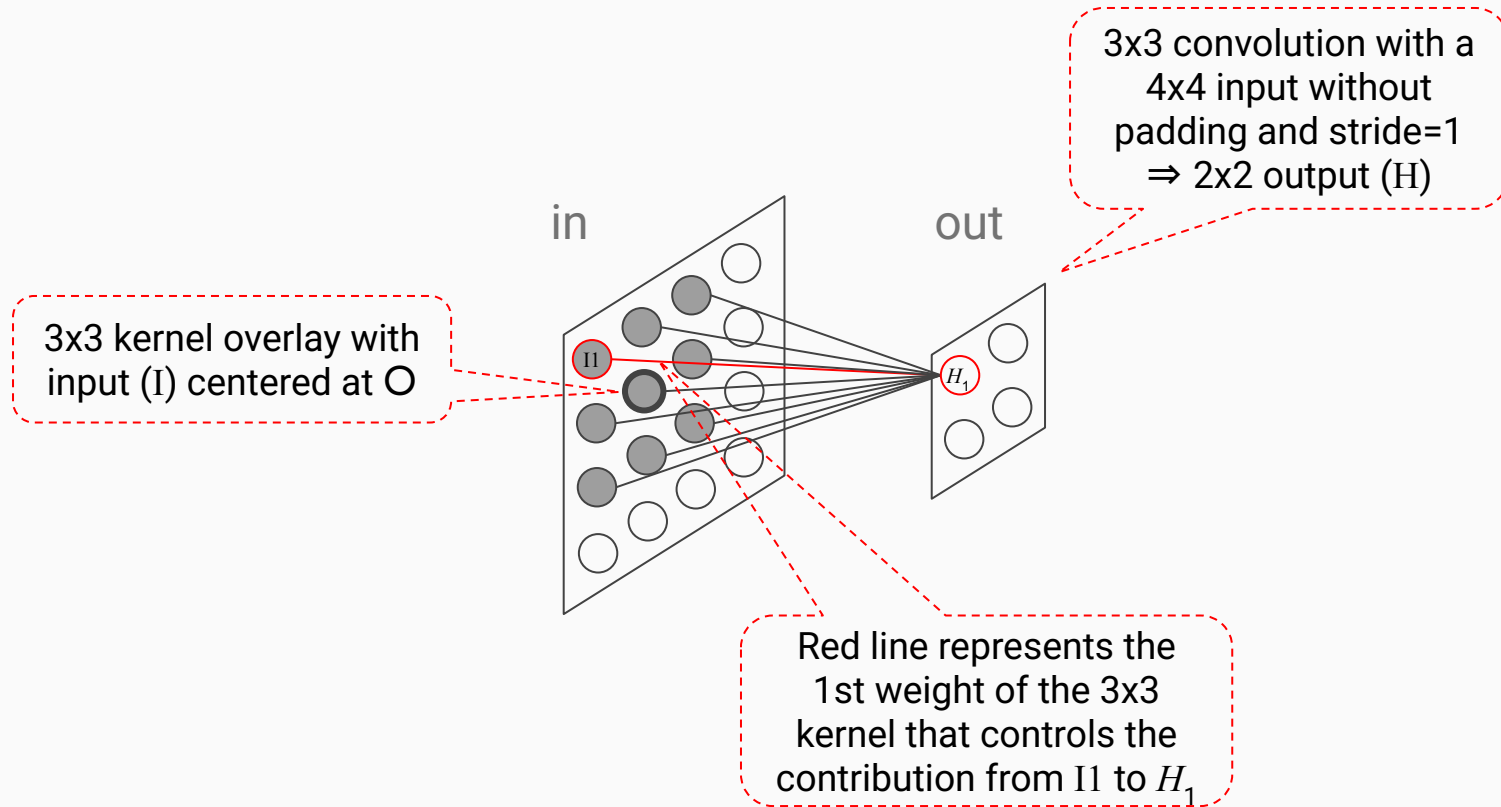
$$\frac{\partial E}{\partial W_{11}^{(1)}} =$$

$$\frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial p_1} \frac{\partial p_1}{\partial h_1} \frac{\partial h_1}{\partial W_{11}^{(1)}}$$

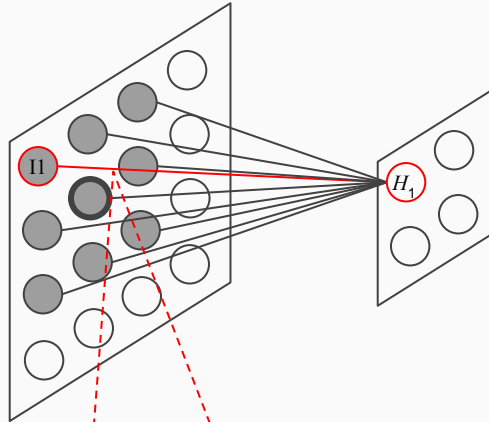
+

$$\frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial p_2} \frac{\partial p_2}{\partial h_1} \frac{\partial h_1}{\partial W_{11}^{(1)}}$$

Convolution with 3x3 Kernel – 1st kernel position

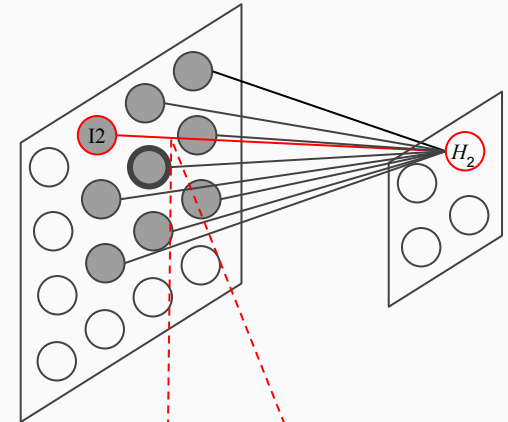


Convolution with 3x3 Kernel – first 2 kernel positions



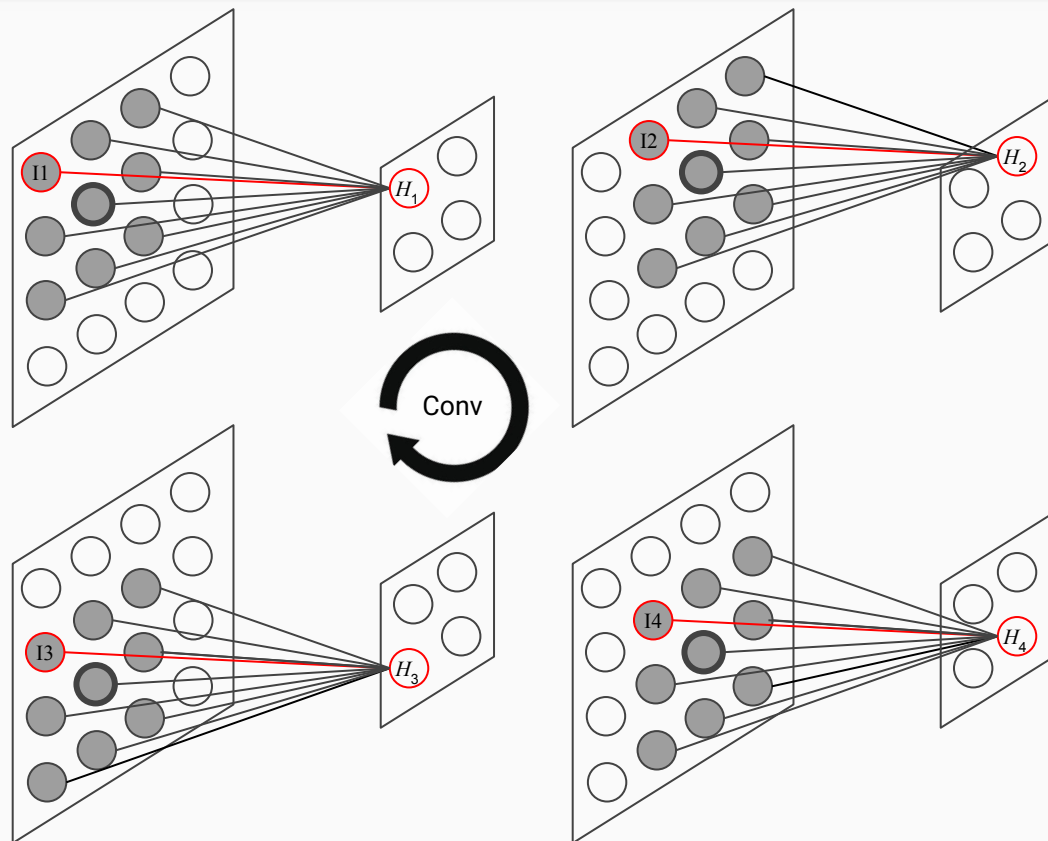
Red line represents the 1st weight of the 3x3 kernel which controls the contribution from I_1 to H_1

Kernel moves to the next position



Red line represents the 1st weight of the 3x3 kernel which controls the contribution from I_2 to H_2

Convolution with 3x3 Kernel – all 4 kernel positions

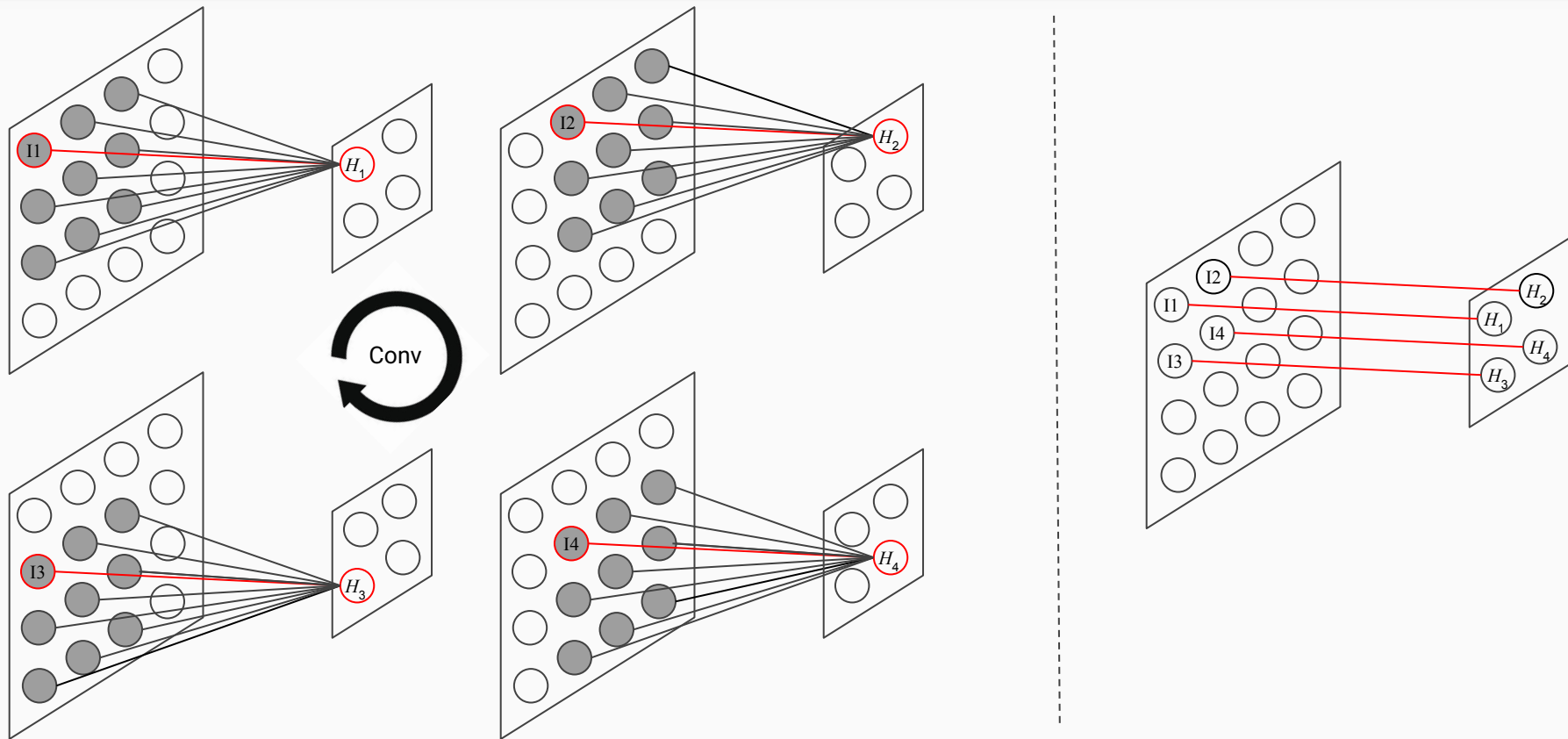


Same weight controls contribution from different input pixel to different output pixel

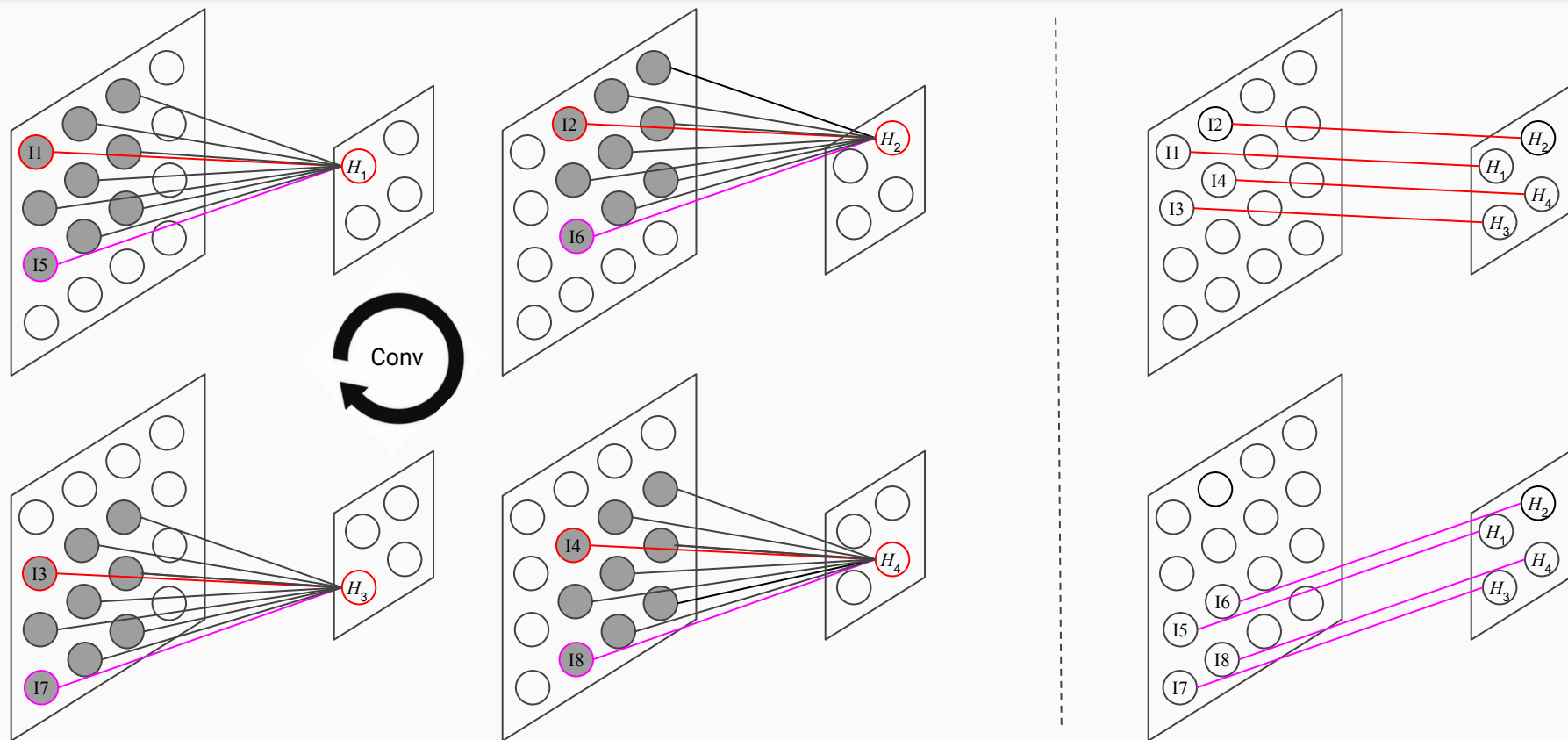
⇒ **Space Invariant**

No matter where the overlay is as the kernel moves around the input image, the kernel weights don't change.

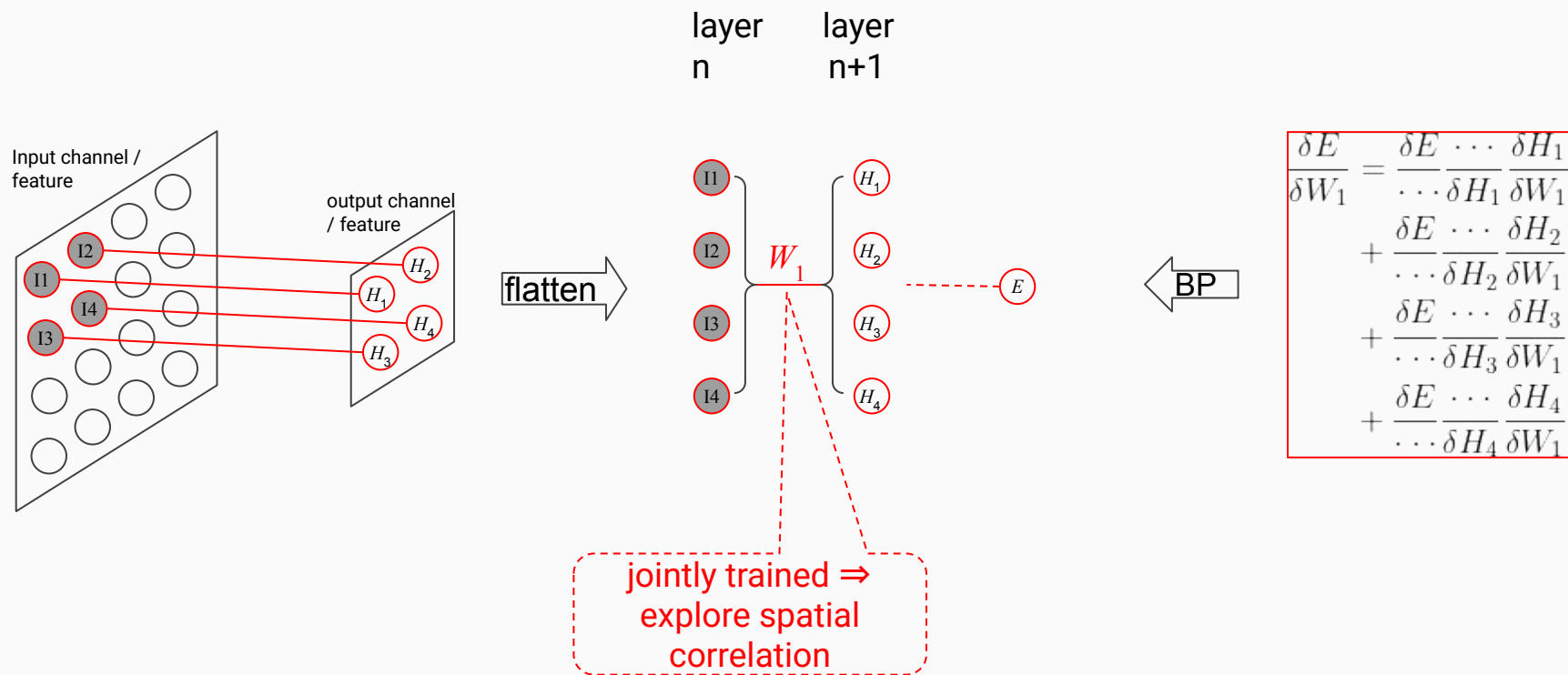
Space Invariant Kernel $\longleftrightarrow$ Same Weight for Many-to-Many Connection



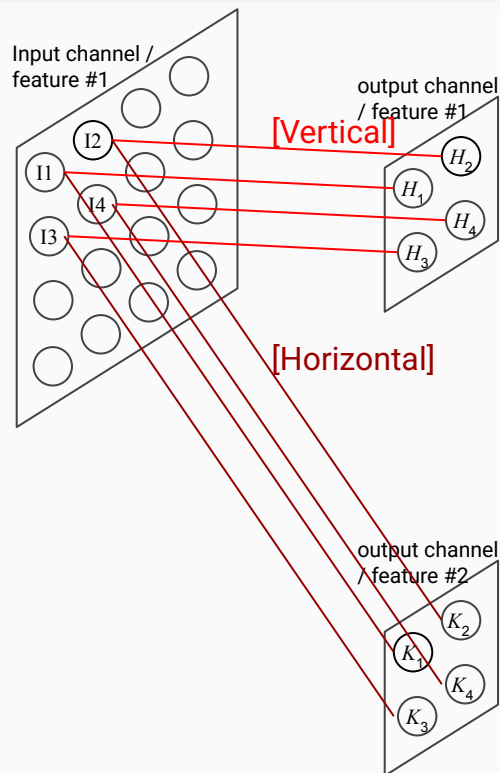
Space Invariant Kernel [another weight in that 3x3 kernel]



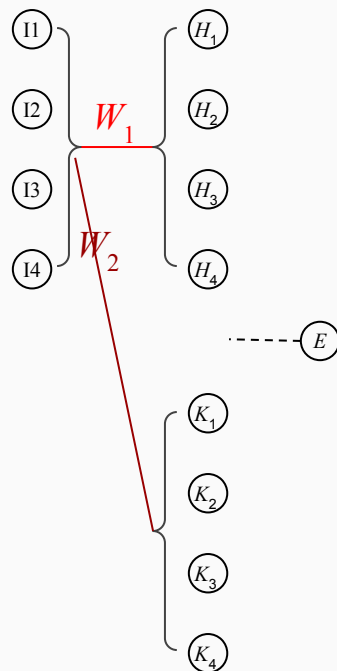
Weight is **Jointly** Trained



Feature Variant Kernel



flatten →

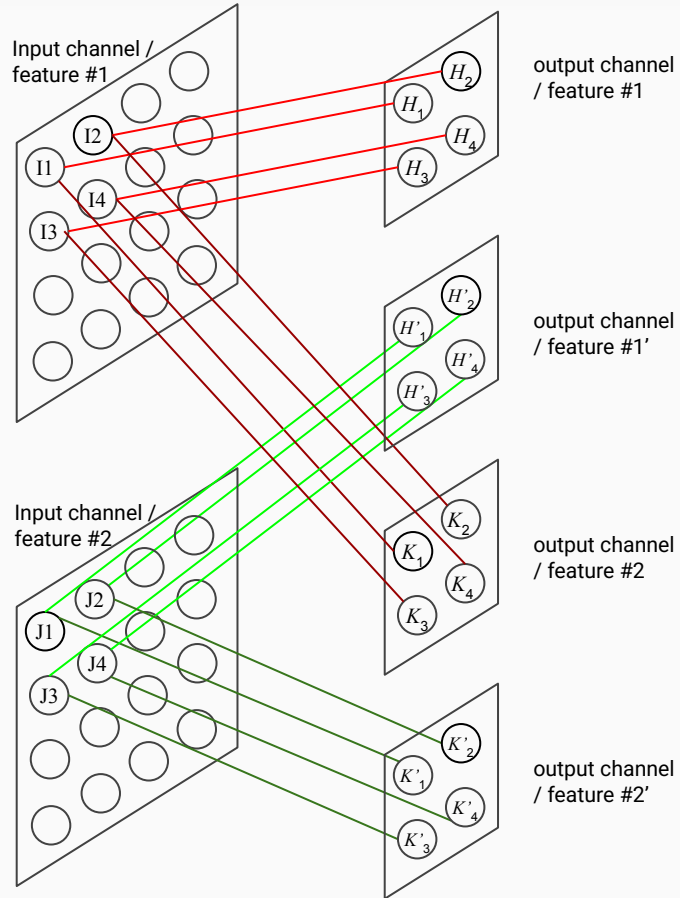


← BP

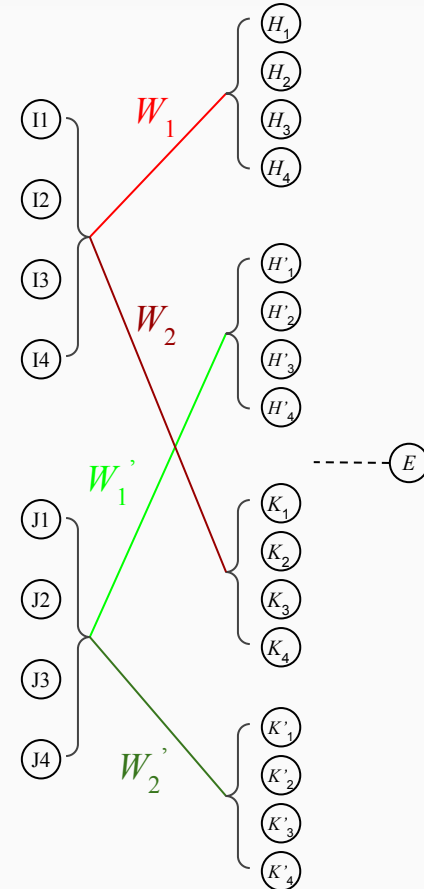
$$\frac{\delta E}{\delta W_1} = \frac{\delta E \cdots \delta H_1}{\cdots \delta H_1 \delta W_1} + \frac{\delta E \cdots \delta H_2}{\cdots \delta H_2 \delta W_1} + \frac{\delta E \cdots \delta H_3}{\cdots \delta H_3 \delta W_1} + \frac{\delta E \cdots \delta H_4}{\cdots \delta H_4 \delta W_1}$$

$$\frac{\delta E}{\delta W_2} = \frac{\delta E \cdots \delta K_1}{\cdots \delta K_1 \delta W_2} + \frac{\delta E \cdots \delta K_2}{\cdots \delta K_2 \delta W_2} + \frac{\delta E \cdots \delta K_3}{\cdots \delta K_3 \delta W_2} + \frac{\delta E \cdots \delta K_4}{\cdots \delta K_4 \delta W_2}$$

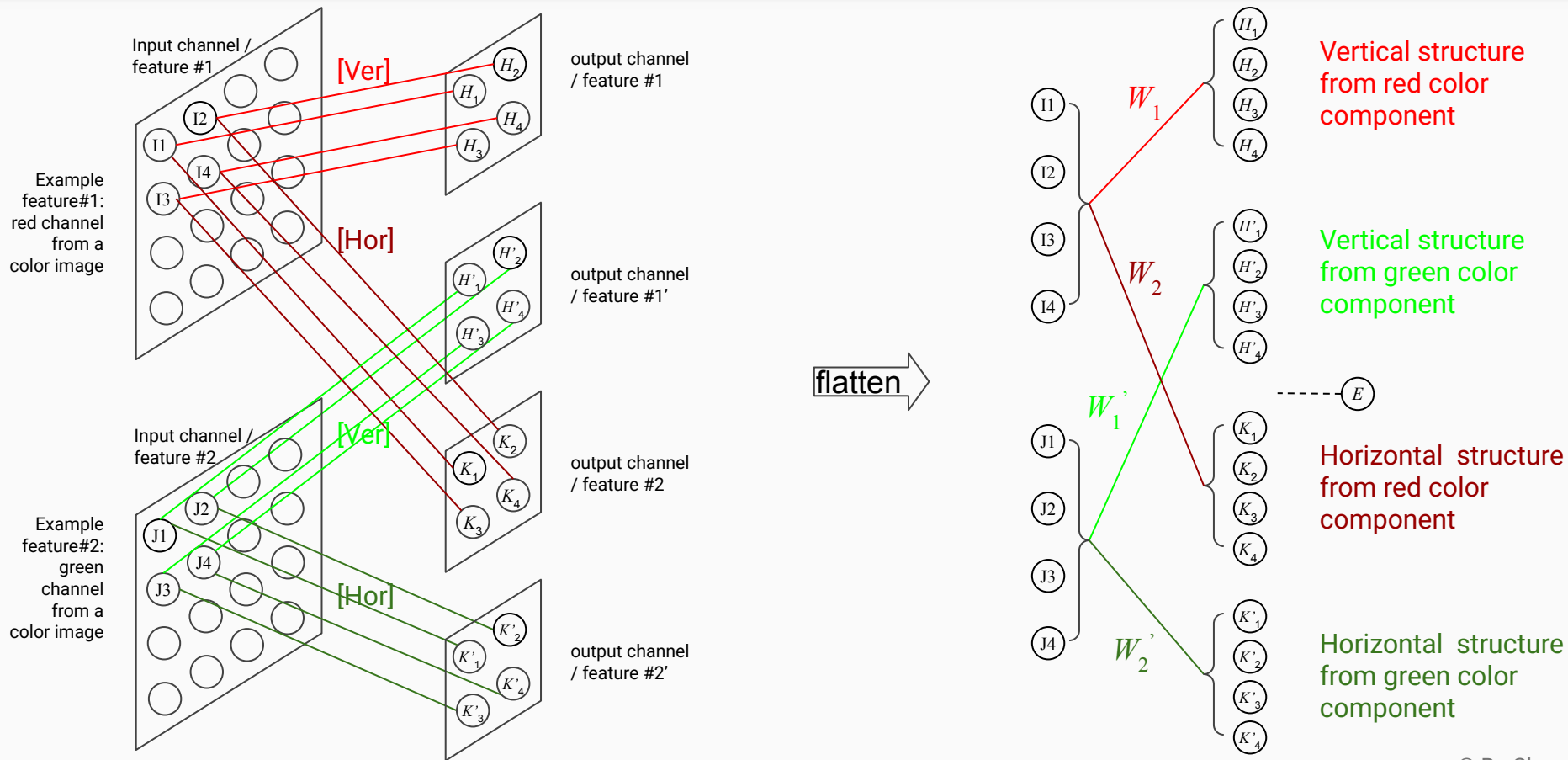
Feature Variant Kernel $\longleftrightarrow$ Combination of In and Out [for illustration only, H' is H , K' is K]



flatten $\rightarrow$



Example Semantic of Each Feature Kernel [for illustration only, H is H', K is K']

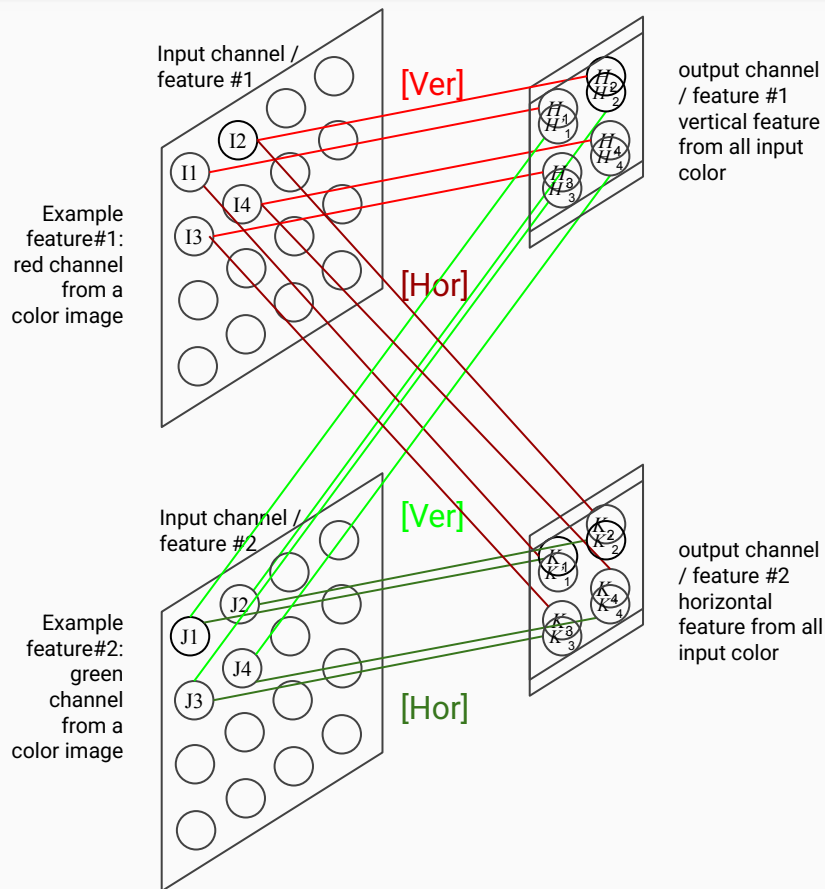


Example Semantic of Each Feature Kernel (Weight Matrix)

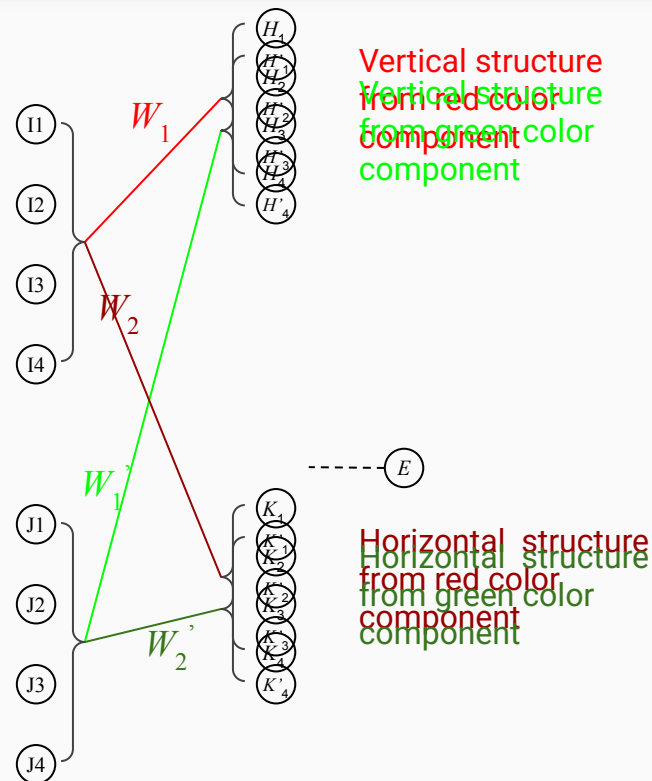
Output Input	Vertical Feature	Horizontal Feature	...
Red Color Channel	Vertical Feature from Red Color Channel ($W1$)	Horizontal Feature from Red Color Channel ($W1'$)	...
Green Color Channel	Vertical Feature from Green Color Channel ($W2$)	Horizontal Feature from Green Color Channel ($W2'$)	...
...	...	...	...

CNN Explainer (Visualizer): <https://poloclub.github.io/cnn-explainer/>
<https://arxiv.org/pdf/2004.15004>

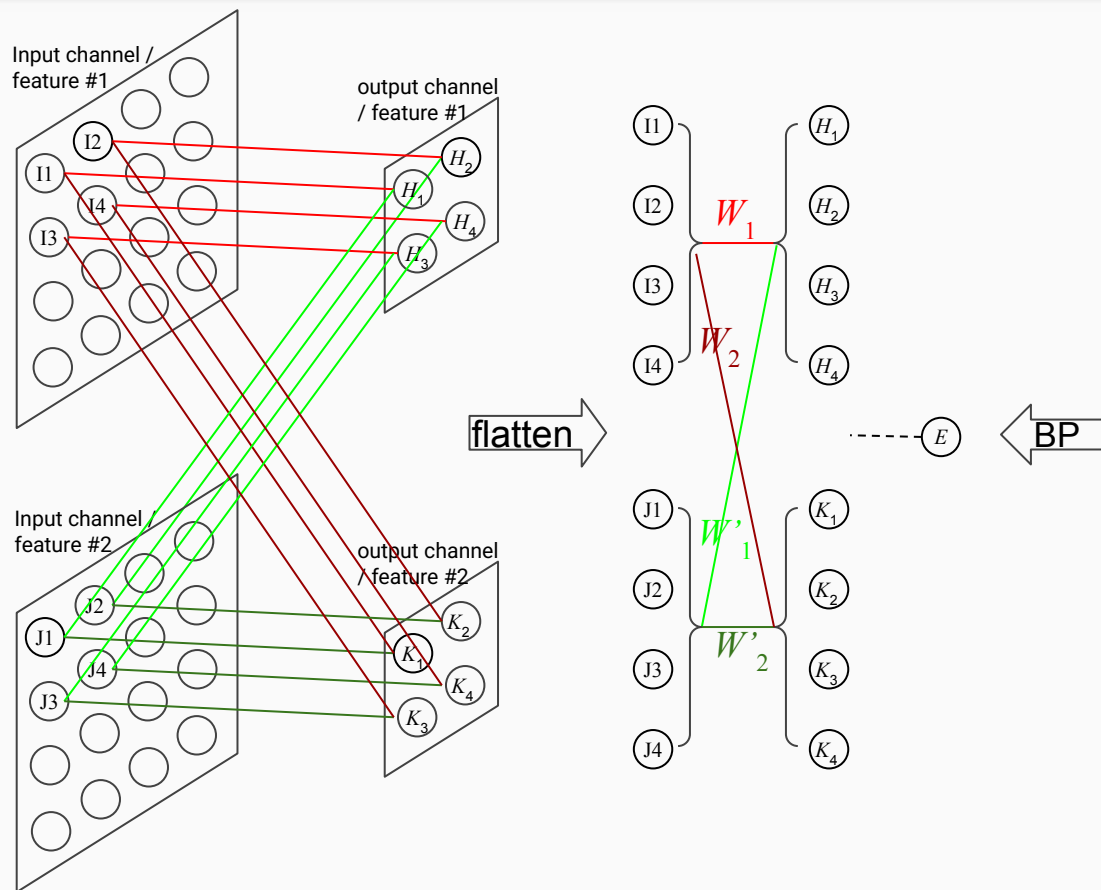
Example Semantic of Each Joint Output Feature [for illustration only, H' is H, K' is K]



flatten



Feature Variant with Back Propagation [Actual Setup]



$$\frac{\delta E}{\delta W_1} = \frac{\delta E \cdots \delta H_1}{\cdots \delta H_1 \delta W_1} + \frac{\delta E \cdots \delta H_2}{\cdots \delta H_2 \delta W_1} + \frac{\delta E \cdots \delta H_3}{\cdots \delta H_3 \delta W_1} + \frac{\delta E \cdots \delta H_4}{\cdots \delta H_4 \delta W_1}$$

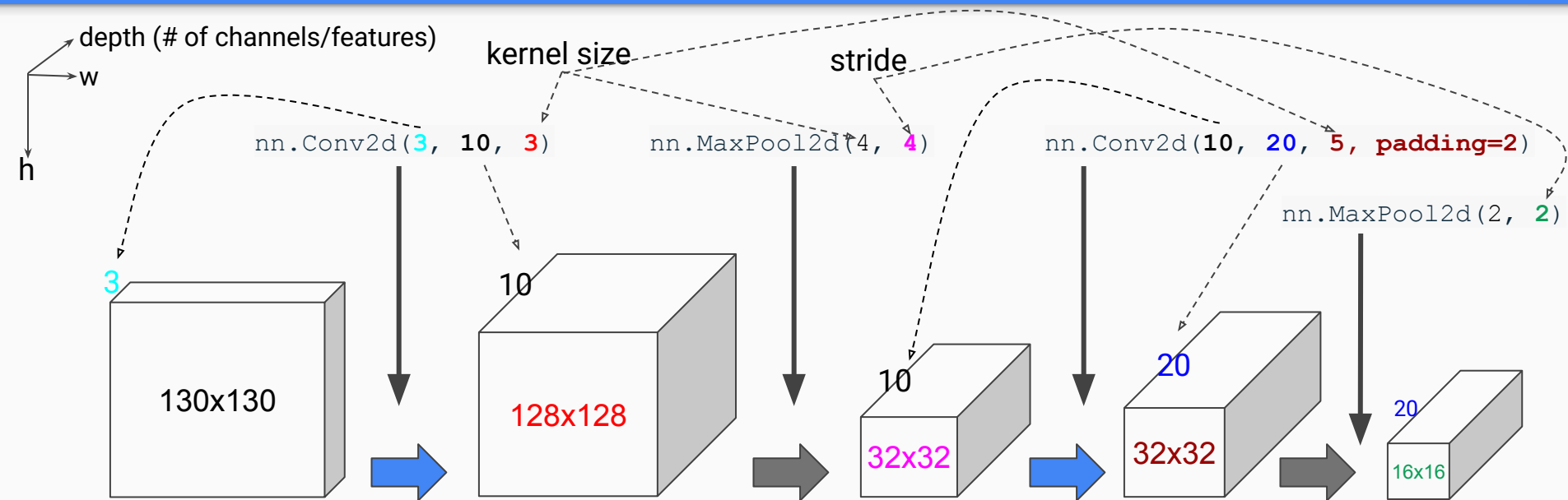
$$\frac{\delta E}{\delta W'_1} = \frac{\delta E \cdots \delta H_1}{\cdots \delta H_1 \delta W'_1} + \frac{\delta E \cdots \delta H_2}{\cdots \delta H_2 \delta W'_1} + \frac{\delta E \cdots \delta H_3}{\cdots \delta H_3 \delta W'_1} + \frac{\delta E \cdots \delta H_4}{\cdots \delta H_4 \delta W'_1}$$

$$\frac{\delta E}{\delta W_2} = \frac{\delta E \cdots \delta K_1}{\cdots \delta K_1 \delta W_2} + \frac{\delta E \cdots \delta K_2}{\cdots \delta K_2 \delta W_2} + \frac{\delta E \cdots \delta K_3}{\cdots \delta K_3 \delta W_2} + \frac{\delta E \cdots \delta K_4}{\cdots \delta K_4 \delta W_2}$$

$$\frac{\delta E}{\delta W'_2} = \frac{\delta E \cdots \delta K_1}{\cdots \delta K_1 \delta W'_2} + \frac{\delta E \cdots \delta K_2}{\cdots \delta K_2 \delta W'_2} + \frac{\delta E \cdots \delta K_3}{\cdots \delta K_3 \delta W'_2} + \frac{\delta E \cdots \delta K_4}{\cdots \delta K_4 \delta W'_2}$$

Visualize Conv/MaxPooling Layers

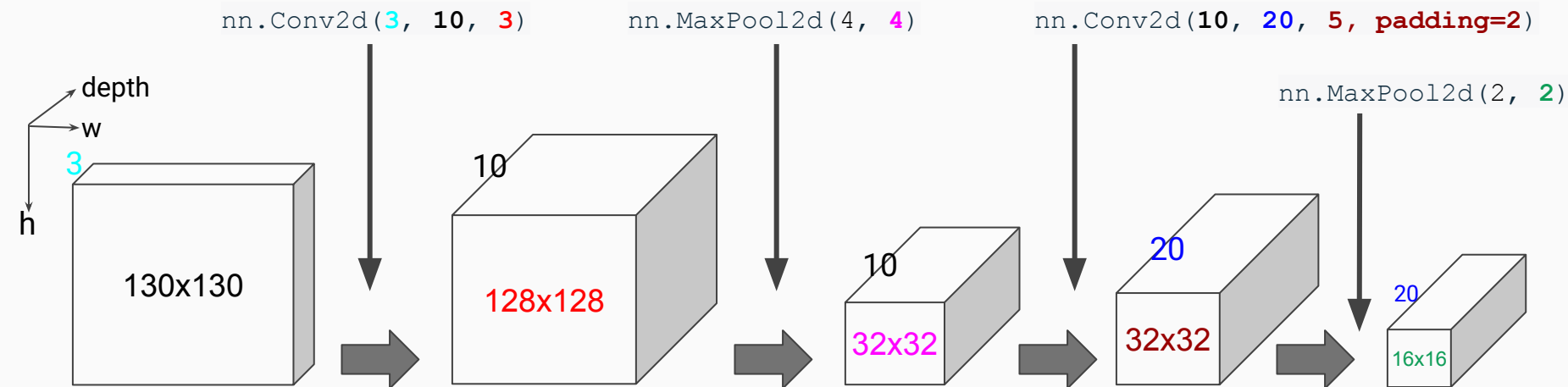
- How many trainable weights (blue arrow)?



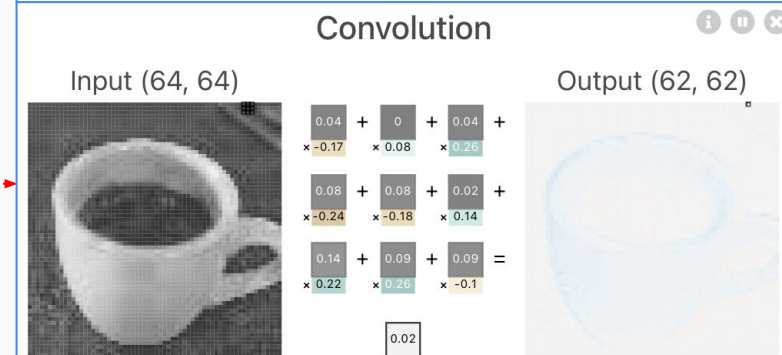
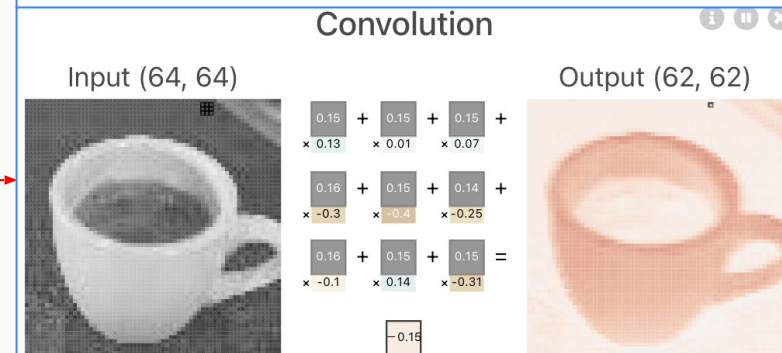
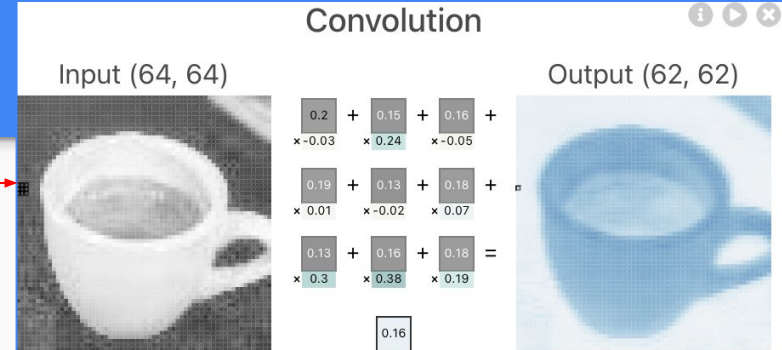
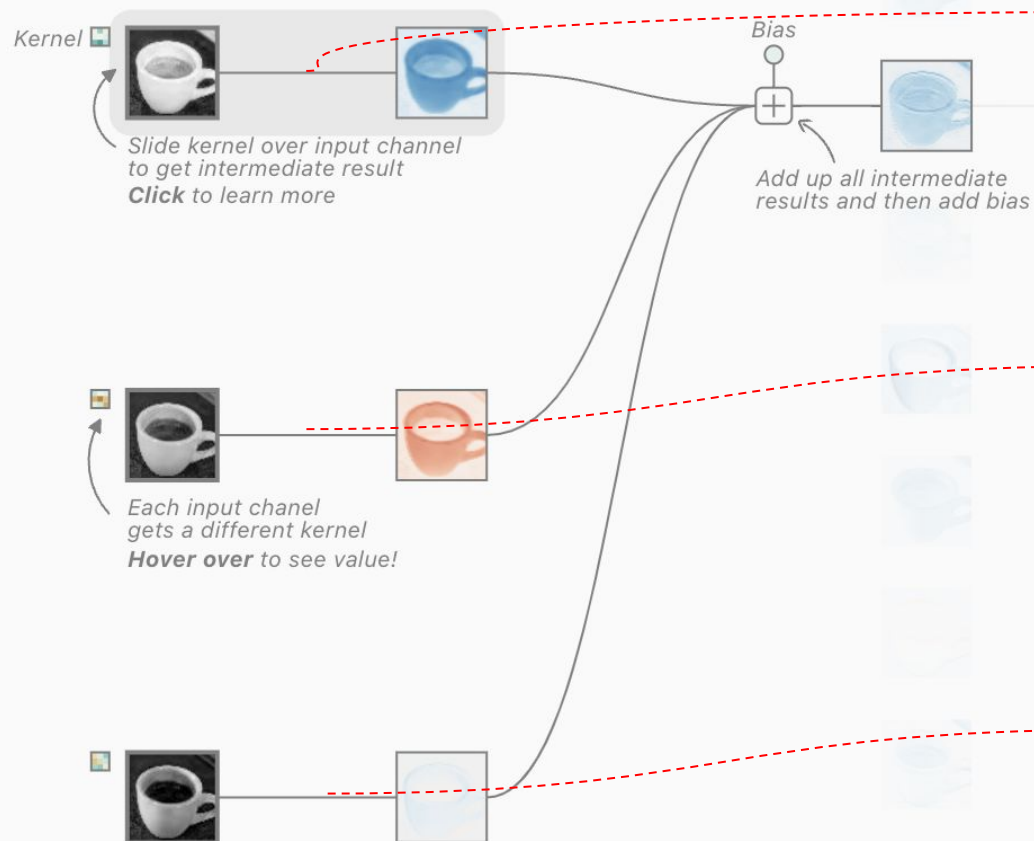
Conv Kernel is **Space Invariant**, but **Channel/Feature Variant**

How many weights to train in the CNN?

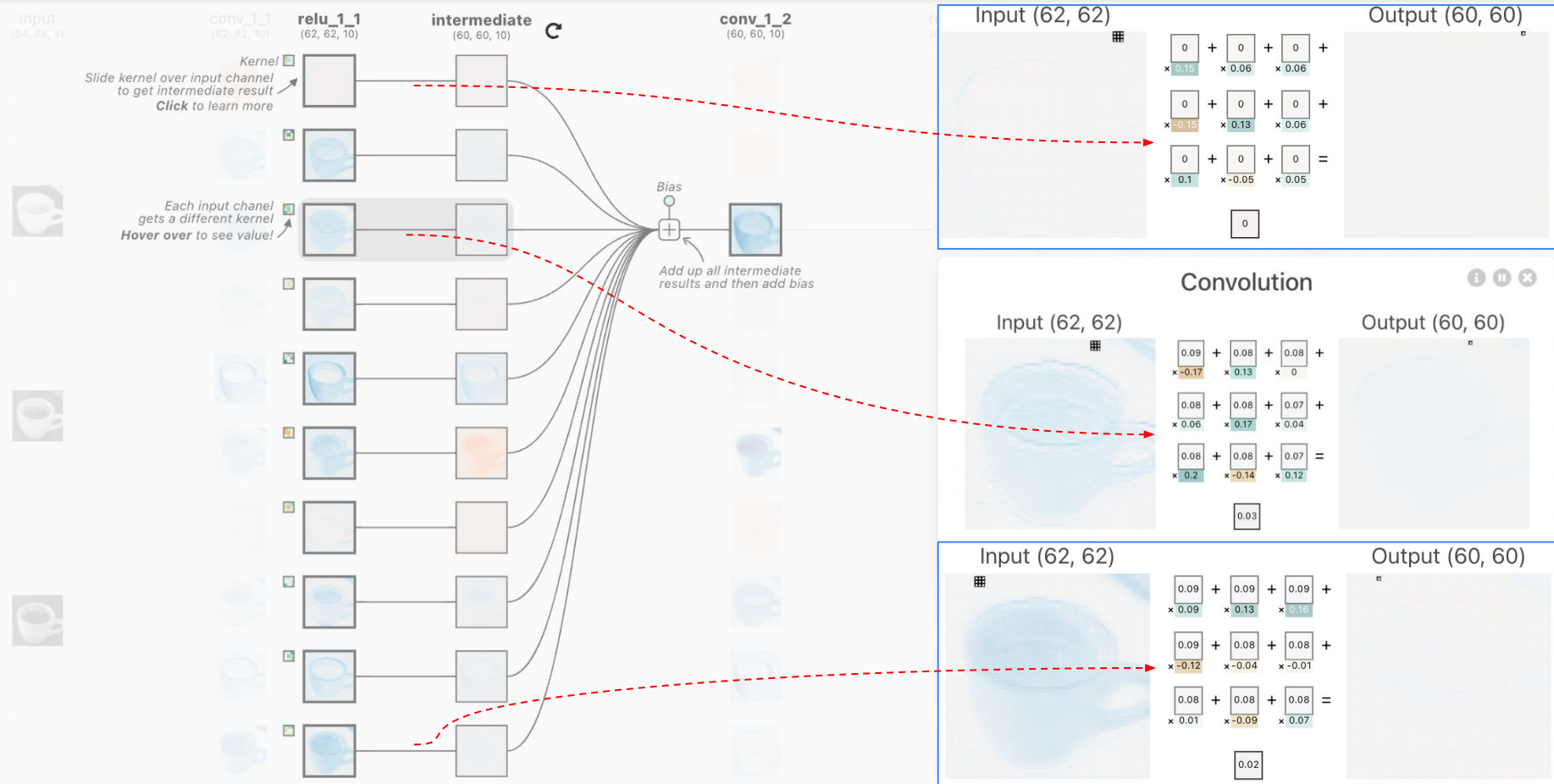
- `nn.Conv2d(3, 10, 3)` $\Rightarrow 3 \times (3 \times 3) \times 10 = \text{inCh} \times (\text{kw} \times \text{kh}) \times \text{outCh}$
- `nn.MaxPool2d(4, 4)` $\Rightarrow 0$
- `nn.Conv2d(10, 20, 5, padding=2)` $\Rightarrow 10 \times (5 \times 5) \times 20$
- `nn.MaxPool2d(2, 2)` $\Rightarrow 0$



CNN Visualizer (1st conv layer, 3 in→10 out)



2nd Convolution Layer, 10 in $\rightarrow$ 10 out [10 kernels for the 3rd output channel in conv_1_2]



More screenshots from:

<https://poloclub.github.io/cnn-explainer/>

Assignment#2: Setup and Practice

- Install **opencv** in your development environments. Here is a reference (old) for MacOS or Linux terminal.
 - If python version 3.7: `pip install opencv-python==4.1.2.30`
 - If python version 3.9.13: `pip install opencv-python==3.4.18.65`
- Practice (no submission required): conventional computer vision approach on feature extraction:
 - Download: Canvas→Modules→JupyterNotebooks→image_filter.zip (it includes an ipynb file and an JPEG image)
 - Complete all the TODO parts (the corner detection part is optional)

Assignment#2: CNN Training & Visualization

- Follow the instructions to build and train your own CNN model
 - You can use the skeleton code as a reference
 - Canvas/Modules/JupyterNotebooks/build_cnn.zip (which includes build_cnn.ipynb)
 - You need to complete all the TODO parts to make it work
- Follow the instructions on the visualization tasks
- You can use Python or C++ (if you can find supporting packages in C++)
- You can use Pytorch (preferred) or Tensorflow.
- You can use Jupyter notebook or Google Colab